

## Introduction

---

There's a particular kind of interview question that looks almost too simple to ask, and that's exactly what makes it dangerous. Maximum Product of Two Digits is one of those. On the surface, it's asking you to find two digits in a number and multiply them. No graphs, no recursion, no dynamic programming table. Just digits and multiplication.

But interviewers don't ask this question to see if you can multiply. They ask it to see whether you reach for the *obvious* solution or the *right* one. Almost everyone's first instinct is to compare every digit against every other digit — a nested loop. It works, and for this specific problem, it's even fast enough. But the moment you generalize this pattern to "find the top K elements in a large stream of data," nested loops stop being harmless and start being the reason your service times out.

This problem is really a disguised version of a much more common interview pattern: **maintain a running top-N while scanning a collection once**. You'll see this exact idea again in problems like "second largest number in an array," "top two salaries," or "top K frequent elements." Learning it here, on a tiny problem with at most ten digits, means you'll recognize it instantly when it shows up somewhere much bigger.

## Problem Overview

---

You're given a positive integer. Your task is to look at its individual digits and find the **two digits whose product is the largest possible**.

A few things to keep in mind:

- You're working with digits (0–9), not the digits' positions or the number itself.
- If a digit appears more than once in the number, you're allowed to use two separate occurrences of it as your pair. For example, in `22`, the digits are `2` and `2`, and you're allowed to multiply `2 × 2`.
- You return a single integer: the highest product achievable from any two digits in the number.

That's the whole problem. No sorting is required, no tricks with negative numbers, no need to worry about digit order — just digits and one multiplication.

## Example

---

**Input:** 124

Digits: 1, 2, 4

Every possible pair and its product: -  $1 \times 2 = 2$  -  $1 \times 4 = 4$  -  $2 \times 4 = 8$

**Output:** 8, because  $2 \times 4$  is the largest product among all pairs.

**Input:** 22

Digits: 2, 2

Only one meaningful pair here:  $2 \times 2 = 4$ .

**Output:** 4

**Input:** 10

Digits: 1, 0

Only pair:  $1 \times 0 = 0$ .

**Output:** 0 — a good reminder that a zero digit anywhere will drag the product down, no matter how big the other digit is.

## Intuition

---

Here's the mental leap that makes this problem click: **you don't actually care about pairs — you only care about the two largest individual digits.**

That's because multiplication of two non-negative digits is maximized by maximizing both factors independently. If you know the single biggest digit and the single second-biggest digit in the number, their product is guaranteed to be the best possible product you can make. You never need to check every combination to know this — it falls directly out of basic number reasoning: to maximize  $a \times b$  where both are non-negative, you want the largest available  $a$  and the largest available  $b$  from what remains.

So the problem quietly transforms from "check all pairs" into "find the top two values in a small list." And finding the top two values in a list is a classic pattern: keep two tracking variables, sweep through the list once, and update them as you go.

Picture a tiny leaderboard with exactly two slots — first place and second place. As each digit walks by, ask:

1. Is this bigger than whoever's in first place? If so, bump first place down into second, and this digit takes first place.
2. If not, is it at least bigger than second place? If so, it slides into second place.
3. Otherwise, it doesn't make the leaderboard at all, and you move on.

By the time every digit has walked past, your two leaderboard spots hold exactly the two digits you need — and you got there without ever comparing a pair.

## Brute Force Solution

---

**Idea:** Try every possible pair of digits and keep the largest product.

**Algorithm:** 1. Convert the number to a list of digits. 2. Use two nested loops — the outer loop picks one digit, the inner loop picks another. 3. Multiply each pair and track the maximum product seen.

**Advantages:** - Extremely easy to reason about and verify correct. - No risk of subtle off-by-one leaderboard bugs.

**Disadvantages:** - Does far more work than necessary — it recomputes information you could have tracked in a single pass. - Doesn't scale. This exact pattern, applied to a much larger list, becomes a real performance problem.

**Complexity:** - Time:  $O(n^2)$ , where  $n$  is the number of digits. - Space:  $O(n)$  to store the digit list.

For this specific problem,  $n$  is at most 10 (a number can have at most 10 digits before it likely overflows standard integer ranges used in these problems), so  $O(n^2)$  is completely fine in practice. But it's worth recognizing *why* it's fine here — small, fixed input size — rather than assuming nested loops are always an acceptable default.

## Optimal Solution

---

The optimal approach drops the nested loop entirely and replaces it with a single pass that tracks the top two digits as it goes.

### Step-by-step:

1. **Convert the number to digits.** Turn the number into a string so you can iterate over individual characters, then convert each character back into an integer.
2. **Initialize two trackers**, `max1` and `max2`, both starting at `0`. These represent your current best and second-best digits.
3. **Sweep through every digit once:**

4. If the current digit is greater than `max1`, then `max1` becomes this digit, and the *old* `max1` gets demoted to `max2`.
5. Otherwise, if the current digit is greater than `max2`, it becomes the new `max2`.
6. Otherwise, it's ignored — it doesn't beat either leaderboard spot.
7. **Multiply** `max1 × max2` once the loop finishes. That's your answer.

Here's a table tracing through 124 :

| Digit | Comparison                            | max1 | max2 |
|-------|---------------------------------------|------|------|
| start | —                                     | 0    | 0    |
| 1     | 1 > 0 → new max1                      | 1    | 0    |
| 2     | 2 > 1 → new max1, old max1 (1) → max2 | 2    | 1    |
| 4     | 4 > 2 → new max1, old max1 (2) → max2 | 4    | 2    |

Final: `max1 = 4`, `max2 = 2`, product = 8. Matches the brute force result, as it should.

The key insight that makes this correct even with duplicate digits: **the demotion rule**. When a new digit beats `max1`, the old `max1` doesn't disappear — it slides into `max2`. That's what keeps repeated digits like the two 2s in 22 from being lost.

## Python Code

```
def max_product_of_two_digits(number: int) -> int:
    """Return the largest product obtainable from any two digits of `number`."""
    max1 = max2 = 0

    for char in str(number):
        digit = int(char)
        if digit > max1:
            max1, max2 = digit, max1
        elif digit > max2:
            max2 = digit

    return max1 * max2

def demo():
    assert max_product_of_two_digits(124) == 8
    assert max_product_of_two_digits(22) == 4
    assert max_product_of_two_digits(10) == 0
    assert max_product_of_two_digits(11) == 1
    assert max_product_of_two_digits(987) == 72
    print("all checks passed")
```

```
if __name__ == "__main__":
    demo()
```

## Code Walkthrough

- `max1 = max2 = 0` — both leaderboard slots start empty (represented as zero, the smallest possible digit).
- `for char in str(number)` — converting the number to a string lets us iterate over its digits one character at a time without manual math (like repeated `% 10` and `// 10`).
- `digit = int(char)` — turns each character back into an actual integer we can compare and multiply.
- `if digit > max1: max1, max2 = digit, max1` — this single line does both the promotion of the new digit *and* the demotion of the old champion, using Python's tuple assignment to avoid a temporary variable.
- `elif digit > max2: max2 = digit` — only checked when the digit didn't beat `max1`; this is where ties with `max1` still get a chance to claim second place.
- `return max1 * max2` — once every digit has been considered, multiply the two leaders.

## Dry Run

Let's trace `987` step by step.

Digits: `9, 8, 7`

1. Start: `max1 = 0, max2 = 0`
2. Digit `9`: `9 > 0` → `max1 = 9, max2 = 0` (old `max1`)
3. Digit `8`: `8 > 9` ? No. `8 > 0` ? Yes → `max2 = 8`
4. Digit `7`: `7 > 9` ? No. `7 > 8` ? No → ignored

Final: `max1 = 9, max2 = 8, product = 72` . 

## Complexity Analysis

**Optimal solution:** - **Time:  $O(n)$**  — each digit is visited exactly once, and the work done per digit (two comparisons, at most one swap) is constant. - **Space:  $O(1)$**  extra — only two integer variables (`max1`, `max2`) are used regardless of how long the number is. (Converting to a string does use  $O(n)$  space, but that's for iteration convenience, not for the algorithm's core memory usage.)

This is correct because the leaderboard invariant holds after every digit: at any point in the loop, `max1` and `max2` are guaranteed to be the largest and second-largest digits seen *so far*. Since every digit gets a chance to challenge the leaderboard exactly once, by the end of the loop they hold the largest and second-largest digits overall.

**Brute force solution:** - **Time:**  $O(n^2)$  — every digit is compared against every other digit. - **Space:**  $O(n)$  — storing the digit list.

## Alternative Solutions

---

A reasonable alternative is to skip manual tracking entirely and use Python's built-in sorting:

```
def max_product_sorted(number: int) -> int:
    digits = sorted(int(c) for c in str(number))
    return digits[-1] * digits[-2]
```

This is shorter to write and easy to read, but it sorts the entire list —  $O(n \log n)$  time — when you only ever need the top two values. For a number with at most 10 digits, the difference is invisible. But it's worth knowing the single-pass leaderboard approach because it generalizes: if you needed the top *three* or top *K* digits, the leaderboard idea extends naturally (or you'd reach for a heap), while re-sorting every time becomes wasteful as *K* and *n* grow.

## Edge Cases

---

- **Single-digit-repeated numbers** ( `11` , `22` , `99` ): the demotion rule correctly lets the second occurrence claim `max2` , giving results like `1×1=1` , `2×2=4` , `9×9=81` .
- **Numbers containing zero** ( `10` , `100` , `205` ): zero digits are still tracked correctly, and if zero ends up in `max1` or `max2` , the final product correctly comes out to `0` .
- **Two-digit numbers** (the smallest valid input, e.g., `10` through `99` ): there are exactly two digits, so the answer is simply their product — no ambiguity.
- **Large numbers with many repeated high digits** ( `999999` ): every `9` after the first two is simply ignored once both leaderboard spots are filled with `9` , and the answer stays `81` .
- **Strictly increasing or decreasing digit sequences** ( `123456` , `654321` ): the two highest digits are always correctly found regardless of their position in the number.

## Common Mistakes

---

1. **Forgetting the demotion step.** Only updating `max1` when a bigger digit appears — without shifting the old `max1` into `max2` — silently loses information and produces wrong answers on repeated digits.
2. **Using strict inequality where a tie should still count.** If you write `digit > max2` but forget it should also fire when `digit == max1` (as in the `elif` structure above), duplicate digits like in `22` won't be handled correctly.
3. **Not converting characters back to integers.** Comparing string characters ( `'9' > '8'` ) technically works for single digits due to how digit characters are ordered, but it's fragile and easy to break — always convert explicitly.
4. **Assuming digit order matters.** Some beginners try to preserve the original position of digits, when the problem only cares about their values.
5. **Using nested loops out of habit** without checking whether a single-pass approach is possible, forming a pattern that becomes a real bottleneck on larger inputs later in your interview prep.

## Interview Questions

---

- How would you modify this to find the **top three** digits and their product instead of two?
- What if the input could contain negative numbers — how does the leaderboard logic change?
- Can you solve this without converting the number to a string, using only arithmetic operations ( `% 10` and `// 10` )?
- How would this approach change if you were streaming digits one at a time and couldn't store them all?
- What's the largest possible input size where the brute force approach would actually become a real performance problem?

## Similar Problems

---

- **Second Largest Number in an Array** — the exact same leaderboard pattern applied to a general list instead of digits.
- **Kth Largest Element in an Array** — a generalization of "track the top two" to "track the top K," often solved with a heap.
- **Two Sum** — while different in goal, it shares the lesson that a single pass with the right tracking structure beats brute-force pair checking.
- **Maximum Product of Three Numbers** — a direct extension of this exact problem's idea to three tracked values instead of two, which is the next problem in this series.

- **Top K Frequent Elements** — another "find the best few without checking everything" problem, just applied to frequency counts instead of raw values.

## Key Takeaways

---

The real lesson here isn't about digits or multiplication — it's about recognizing when a problem is secretly asking "find the top few values in a single pass." Once you see that pattern, the nested-loop instinct disappears, and you reach directly for two (or K) tracking variables updated as you scan. That one shift in thinking — from "compare everything" to "track the best as you go" — is one of the most reusable ideas in all of coding interviews.

## Watch the Video

---

If you'd rather see this traced out visually, step by step, with the leaderboard analogy animated in real time, check out the full walkthrough here: {LINK}

## About the Series

---

This problem is part of the **Daily Python LeetCode** series, where a new problem is broken down every day — starting from the intuition, moving through brute force, and landing on a clean, optimal Python solution. The goal isn't just to solve today's problem, but to build a toolbox of patterns you'll recognize instantly in future ones. Tomorrow's problem builds directly on this one, extending the same idea to three numbers instead of two.

## Call To Action

---

If this walkthrough helped the idea click, give the video a like on YouTube and subscribe so you don't miss tomorrow's problem. For the full written breakdowns like this one delivered straight to your inbox, subscribe to the Substack newsletter. And if you know someone prepping for interviews who's stuck overusing nested loops, share this with them — and drop a comment with what number you'd want traced through next.

## Quick Review

---

**What pattern fits 'find max product of two elements' scans?**

Single-pass tracking of top-K values — keep top 2 seen so far while scanning once.

**Time complexity of the top-two-tracking approach?**

$O(n)$  — one pass through the array, constant work per element.

**Space complexity of tracking top two digits?**

$O(1)$  — only two variables (largest, second-largest) needed, no extra array.

**Common bug when updating top-two trackers?**

Overwriting 'largest' before checking against 'second-largest', losing the old max — update second first.

**Brute force vs top-two-tracking: what's the tradeoff?**

Brute force checks all pairs in  $O(n^2)$  but is simpler; tracking top two is  $O(n)$  but needs careful update order.

**Interview follow-up: what if array has duplicate max values?**

Still works — track by index or count occurrences so the same element isn't paired with itself.